A Novel AI-Augmented Programming Language Design

Improving Software Development by Flexible Controllability in Program Design and Coding Granularity on Structural Oriented Syntax and Natural Semantics Understandability

Yujun Kong
Stevens Institute of Technology
ykong3@stevens.edu

Abstract

This proposal outlines the conception and design of a new AI-enhanced programming language, explaining how it was conceived and how it is to be designed to overcome the significant shortcomings of existing software development tools that currently rely on AI. The language facilitates greater control over the granularity of coding using Natural Language Semantics and offers increased freedom in program design. It is, in turn, inspired by markup languages and incorporates artificial intelligence to make code generation smoother, creating a means of controlling program logic and maintaining transparency in the interconnection among software components. The investigations follow two approaches: on the one hand, the study is based on the qualitative assessment of AI-supported technologies, and the other hand, the quantitative analysis of a layer structure of software technologies and the complexity of AI-aided compilation are explored. Multiple mathematical models are suggested, with one that tracks the weighted semantic process behind AI-assisted code generation. In addition, an algorithm is proposed to specify the programming granularity weighing mechanism. This work aims to substantially enhance the effectiveness, accuracy, and structured handling of complex software systems utilizing AI in an advanced manner.

Keywords: Programming Language Design, Artificial Intelligence (AI), Natural Language Processing (NLP), Software Component, Coding Granularity, Natural Semantics Understanding, Compilation

Motivation

Although the world is in a great recession, the fast pace of development of human society's technologies never hesitates; in particular, the momentum in the technological evolution of artificial intelligence (AI) is unstoppable. So, with our deepest humbleness in the greatest interest of participating in this AI era, we are thus wondering if there is still a profoundly brand-new or game-changing achievement that we can do to better this lovely but rigorous tech world in Computer Science.

In recent days, we have seen a tremendous number of AI-driven or AI-assisting tools or platforms becoming increasingly popular, getting more and more acceptance from ordinary people as end-users and professional practitioners as research scientists. Especially in the applicable areas of Software Development, some are legitimately excellent in performance on real-time implementations of software program design and functionalities feature delivery, such as 'Microsoft Copilot', 'Amazon CodeWhisperer', 'Devin', etc., and we highly admire these newly revolutionary arrivals in the field with their amazing AI-enhancing characteristic. However, we somehow noticed that, in these AI-powered applications of Software Development, their program design patterns and usability organizational mechanisms, specifically, the coupling/linkage of the software modules implementation, there are still a bunch of more or less under-developed aspects lacking feasible control from coding quality and components design in coherency, consistency, calibrations, and collaborations.

Therefore, here we elicit an idea according to the above-mentioned qualitative analysis of the AI-assisting applications of the Software Development industry today: Can we develop (invent) a brand new Programming Language that is capable of not only supporting the features in Natural Language Semantics Prompting Understandability by the compiler but also featuring control of the implementations on any level in Programming Granularity Flexibility. This is exactly the purpose of the research.

Introduction

- **Background, Inadequacy, and Elicitation**

  As the former chapter noted, there's a never-before inclination that the whole technology world is increasingly indulging in a mania of investing and committing to research and breakthroughs in Artificial Intelligence (AI). With that, we see countless applications embedded in various AI technologies. We must admit that a considerable number of them are revolutionary and excellent, far beyond the expectations of the people once believed, which exactly reflects on the performance in aspects of easy-use experience, content generating velocity, and highly satiated with the generative results. Yet, as time went by, discrepant voices with negative opinions iteratively emerged from the public, especially the professionals in the academia of Computer Science and the industry of Software Development/Engineering.

  Ecumenically, the people as the heavy or deep users who tried leveraging this AI-assisting stuff to the realistic industrial applicable areas somehow felt stuck and had difficulties; the AI-generated outcomes, though seemingly matched in general, were still precisely unqualified to be suitable for applying to solve the real problems. To the experienced senior-level software developers or researchers, especially in industrial fields, regarding their attitudes toward commenting AI-embedded tech traits, stated that these AI tools/platforms (cloud or local Integrated Development Environment), in systematically building specific software components and structurally implementing arbitrary given coding units, do not perform such good and even somewhat badly. In reality, at some point when a developer, in the holistic program design consideration, likes to actualize one software component with the feature that requires coupling collaboration by two or more units/modules (classes or objects acting as code snippets), the automatic AI-generative coding mechanism seems don't apprehend the real intent from the developer, deviated to result in inconsistency on program design or detoured to generate irrelevant codes redundantly. And sometimes, more importantly, when all the AI-generated program units have to be functional together, the software system performs oddly and even fails because of the existing logic bugs.

  According to the above circumstantially, not a few developers, researchers, scholars, and scientists in Computer Science and Software Engineering criticize that, at least currently, the most popular AI-powered Software Development applications might still be incapable of challenging the work on building the sophistically designed software applications, not to mention the High-Performance demanded industry-level software systems. "'Artificially' conducting a bunch of toys," said one of our former instructors, Professor Dr. Kempinski, at the Stevens Institute of Technology, which was pretty sarcastic and awkward. So, there's a naturally brought question confronting us: Do we have the possibility to develop a brand-new State-of-the-Art Programming Language that could be not merely more advanced in championing to generate high-quality codes based on the understanding of the prompted Natural Semantics but meanwhile enabling to secure the controllability in flexibility freely on any level's programming granularity in holistic program/components design and software features implementation? Of course, for now, we don't yet believe it can be materialized shortly. Still, the answer remains affirmative regarding the feasibility of this novel Programming Language. Undoubtedly, this is the right pacing way to lead us to contribute worthy commitment and dedication to bring the never-before convenience, effectiveness, efficiency, precision, power, and, more significantly, the feasibility of producing real-world applicable industry-level software applications/systems.

- **Rationale and Mechanisms**

  Stemming from our long-term leveraging of existing applicational technologies, contemporary academic research achievements, and industrial work experience related to Programming Languages and Compiler Designs, we fathomed that a Marking Language, like HTML or XML (Hyper-Text Markup Language or Extensible Markup Language), for building software units/entities features a structural way in design and implementation may exactly match the characteristic which owns the controllability and flexibility in programming granularity among the "big picture," units/components, and specific tiniest coding chunks in realistic tasks of Software Development projects. With that, we do believe this

novel Programming Language, with the feature of AI-powered Natural Semantics understandability in Syntax, could be simple to the existing Markup Languages such as HTML and thus predesigned as follows code patterns when acting program designs and implementations in Software Development tasks:

Figure 1: Known Syntax of HTML

```
<html>
  <head></head>
  <body>
    <h1>Hello World.</h1>
      <div>
        <a href='http://www.github.com/
            stevens_go_ducks'>
          GoDucks GitHub Repos
        </a>
      </div>
  </body>
</html>
```

Figure 2: Conceived Syntax of the Language

```
<app scope='compunit'; AI: A form,
    users do data analysis; ProgLang:
    Java; UI: JavaFX>
  <module feature='a bunch of selection
      items'; compunit='
     analytic_method_wsj'>
    <ai GenCnt: social media's
        commenters, opinions, negative>
    </ai>
  </module>
  <module>
    <aigc>
      Plot a data visualization, using
          Matplotlib
    </aigc>
  </module>
</app>
```

From the above two exemplified code snippets, respectively, in HTML and our newly supposed Programming Language, we can, at a glance, briefly apprehend the design thinking of this novel Coding Language, which is how we experimentally adopt the particular approach in structurally handling the program (coding/programming) granularity practicalities of Markup Language to this our novelly-conceived Programming Language featuring its AI-augmented code-generative capacity. We see, by mimicking the syntax of HTML or XML, hypothetically in Figure 2 as a code snippet, we implemented a series of software features that were monolithically programmed like a single software component working out as a user-interfaced application module that provides some data analytics and its data visualization functionalities for the users. According to the code review, we see a variety of tagging pattern keywords behaving like XML (Extensible Markup Language) elements such as <app>, <module>, <ai>, and <aigc>, and each pair of the combinations in open and close tagging like <ai> and </ai> is intended to respectively convey the relevant meaningfulness for compilations to the on-coding component and in unison logically assembles the whole program based on the very structural characteristic in program design and software features achievability.

So, in further detail, we evidence that the pair tags <app> and </app> act as the open and end keywords that scope the presently implementing program unit or source file in the exact ongoing development work of the software application project. In addition, the parameters among the tag <app> act as the description that provides the crucial possibility to let the compiler know what exactly the program or component looks like in software features actualization as our general software and focused program design thoughts, which the description can be quite brief or precise, or even remaining blank circumstantially in the specification. For instance, differently, where the term "feature='a bunch of selection items, choosing the existing methodologies'" in the tag <module> indicates that depending on the understandability of natural language semantics via the AI-augmented mechanism, the compiler shall apprehend the meaning of our stated description and thence properly generate the qualified codes that implement the software feature. Furthermore, the In-key-valued in describing ways like "ProgLang: Java;" and "UI: JavaFX;" represent our required programming purpose that we command the compiler to Artificial Intelligently generate the codes of Java Programming Language and that ought to be restrictively conducted by the framework of JavaFX in user interface implementations.

With that, before, since the delineation for programming keywords on the language trait of structured interpretation so that here we hope your deep fathoming of the design thinking of this in-proposal introduced an AI-augmented Programming Language by the following explicitly demonstrated generative output from the compiled code snippet in Figures 2 to 3:

Figure 3: Selected AI-generated Codes by the Language's Compilation on the Client-side

```html
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="
        width=device-width, initial-
        scale=1.0">
    <title>AI-PL Application Interface
        </title>
    <script src="https://cdn.
        tailwindcss.com"></script>
    <style>
       body { font-family: 'Inter',
           sans-serif; }
    </style>
</head>
<body class="bg-gray-100 p-8 flex
    items-center justify-center min-h-
    screen">
    <div class="bg-white rounded-lg
        shadow-xl p-8 max-w-2xl w-full">

        <h1 class="text-3xl font-bold mb
            -6 text-center text-gray-800
            ">Data Analysis Application</
            h1>

        <!-- Module 1: Social Media
            Analysis -->
        <div id="analytic-module" class=
            "mb-8 border-b-2 pb-4">
            <h2 class="text-2xl font-
                semibold text-gray-700 mb
                -4">Social Media Opinion
                Analysis</h2>
            <div class="mb-4">
                <textarea id="comment-
                    input" class="w-full h
                    -32 p-3 border rounded-
                    lg focus:outline-none
                    focus:ring-2 focus:
                    ring-blue-500"
                    placeholder="Enter
                    social media comments
                    to analyze..."></
                    textarea>
            </div>
            <button id="analyze-btn"
                class="w-full bg-blue-600
                 text-white font-bold py
                -3 px-6 rounded-lg hover:
                bg-blue-700 transition
                duration-300">Analyze
                Opinions</button>
            <p id="analysis-result" class
                ="mt-4 text-gray-600 text-
                center font-medium"></p>
        </div>
        <!-- Module 2 -->
        <div>...</div>
    </div>
</body>
</html>
```

Figure 4. - Selected AI-generated Codes by the Language's Compilation on the Backend-side

```java
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.util.Random;

import com.sun.net.httpserver.
    ↪ HttpExchange;
import com.sun.net.httpserver.
    ↪ HttpHandler;
import com.sun.net.httpserver.
    ↪ HttpServer;
import org.json.JSONObject;

public class MainApp {
   public static void main(String[]
       ↪ args) throws IOException {
      HttpServer server = HttpServer.
          ↪ create(new
          ↪ InetSocketAddress(8000),
          ↪ 0);
      server.createContext("/api/
          ↪ analyze", new
          ↪ AnalyzeHandler());
      server.createContext("/api/plot",
          ↪  new PlotHandler());
      server.setExecutor(null);
      server.start();
      System.out.println("Server
          ↪ started on port 8000.");
   }

   // For the first module.
   static class AnalyzeHandler
       ↪ implements HttpHandler {
      @Override
      public void handle(HttpExchange
          ↪ exchange) throws
          ↪ IOException {
         if (!"POST".equals(exchange.
             ↪ getRequestMethod())) {
            exchange.
                ↪ sendResponseHeaders
                ↪ (405, -1);
            return;
         }

         InputStream is = exchange.
             ↪ getRequestBody();
         JSONObject requestBody =
            new JSONObject(new String(
                ↪ is.readAllBytes()));
         String text = requestBody.
             ↪ getString("text");
         is.close();

         Random random = new Random();
         double sentimentScore =
             ↪ random.nextDouble();
         String sentiment =
            sentimentScore > 0.5 ? "
                ↪ positive": "negative
                ↪ ";

         JSONObject responseJson = new
             ↪  JSONObject();
         responseJson.put("sentiment",
             ↪  sentiment);
         responseJson.put("score",
```

```
                ↪ sentimentScore);
45
46          String response =
                ↪ responseJson.toString()
                ↪ ;
47          exchange.getResponseHeaders()
48                .set("Content-Type", "
                      ↪ application/json"
                      ↪ );
49          exchange.sendResponseHeaders
                ↪ (200, response.length()
                ↪ );
50          OutputStream os = exchange.
                ↪ getResponseBody();
51          os.write(response.getBytes())
                ↪ ;
52          os.close();
53      }
54  }
55
56  // For the second module...
57 }
```

The codes as a program component in Figure 3 and 4 are the executed outputs of the mutual generative efforts from both leveraging the AI Natural Language Processing capability in feasibly handling the software developer-prompted statement tokens as the attributes-parameters and keys-values definitions in a human language semantics description way, and the targeted program design and implementation by our designed programming language that obsesses the power in controlling granularity in structurally programming design like when we code in HTML or XML on considerations of levels and hierarchies in the holistic program thinking for the implementations of the software, components, units, and even code-fragments. Following the continuous observation, we learned the reason why it was programmed in Java is that, in the former example code snippet of Figure 2, we explicitly pointed out our designed programming language's compilation in NLP mechanism, on coding the program, shall artificial-intelligently generate the codes in Java only for the finalized step in compilation to actualize the software features, which the prompted key-value description in natural human language semantics "ProgLang: Java" exactly reflected this very mechanism in rationale; meanwhile, another expression in the key-value definition "UI: JavaFX" too means the same.

Therefore, through the former elucidations, we could fundamentally realize that, intrinsically, the encapsulated Attributes-Parameters and Keys-Values expressions in the markup tags are just like the availability of the prompts that the end users input in the existing Artificial Intelligence-assisting or driven applications in Software Development. Nonetheless, there are still some discrepancies between the two, particularly in performances of turning the prompts into AI-generative codes that could properly apply to not only the sequel implementations in software features but also the better integration of software components, coupled with high consistency and coherence. In other words, although they are the same things in an AI-driven prompting-thence-generating mechanism, the approach of applying this pioneeringly conceived programming language performs far better on the interoperability of respectively coding implemented but functionally coupling software components because of the controllability in the logic of program design and flexibility of coding granularity on the big picture and exactitude iterations. Thus, naturally, we cognized this very idea of blending the applied AI technology, particularly in Natural Language Processing (NLP) competence of semantics understandability, into our elaborately designed structural programming language, which features the power of handling the implementational granularities and accuracies is a tenable solution to encounter the competences and challenges tomorrow in Software Development expertise with the Artificial Intelligence use.

To recap, the above writing of the chapter Introduction to this research proposal may only partially unveil and ultimately explain the concept and all atomic-level details with great precision on superbly qualified thesis development. Yet, as the mini version of a research topic proposal, especially on the Programming Language Design sub-topic, it has already been foreseeably achieved as a highly applicable start in technological critical thinking on elicitation for the world of Computer Science and Software Engineering. However, as the "prophet," we must foresee that there are still considerable and pending researchable blanks waiting for our successive intensive work.

Related Works

Studying the related works plays a critically crucial role in smoothly driving our work to good enough

scholarly standards and industrial applicability on the judgment of verifiability, tenability, and final success in the general scope of the research. Yet, for this research proposal or the planned formal paper later, its pivotal responsibility is first to fathom the contemporary technological development situation and achievements on the research topic as well as ours, thence identifying their work's insufficiency and defects, which means seeking and getting the research gap, and raising our solution on changes and refinements, and ultimately proving why our research ought to be worth developing and would be valuable for the academic and industries. Our literature review authoring will focus on three kinds of research inclinations from formal papers or casual writings relevant to our work scope to strengthen the research:

1. What are the opinions on how the present state-of-the-art AI-assisting software development applications impact the world, from ordinary users to professional developers?
2. What are the analyzed and evaluated outcomes of the comprehensive performance of having leveraged the current trendy AI-powered Software Development applications conducted by experts and scientists?
3. What are the thoughts of the Software Development/Engineering practitioners about the foreseeability and feasibility of designing and actualizing an AI-augmented Programming Language with the Integrated Development Environment?

Since this proposal is merely an initial work of the research as the beginning for our later intensive research work, thus we are not going to organize the context structure rigorously but loosely follow the natural authoring mindflow to elucidate how we comprehended the presently existing phenomena and achievements in AI-assisting Software Development, what the research gap we recognized looks like based on our analysis and evaluation and what are we supposed to dedicate to fulfill it.

Not to mention, around the whole world, the vicious momentum of speedy development and advancing revolution in applying Artificial Intelligence (AI) and exploration for Artificial General Intelligence (AGI) in broad reality, we have evidenced that the strategic taking advantage of current AI technological competence in applications development is never-before reshaping the field of Software Development. The impact is ubiquitous and profound, spanning improved productivity, enhanced processes, secured outcomes, optimized efficiency, and user experience when people, whether common users or professional developers, undertake their Software Development tasks.

- **Contemporary Perspectives**

  From a user perspective, AI-assisted applications significantly enhance accessibility and usability by providing adaptive and user-friendly interfaces. These tools streamline complex tasks, reduce cognitive load, and enable individuals with disabilities to interact more effectively with technology through personalized functionalities [1] (Eziamaka et al., 2024). Such advancements foster a more inclusive digital environment by offering solutions tailored to diverse capabilities and preferences. Notable innovations include speech recognition and natural language processing, which support accessibility for users with physical or cognitive impairments. However, concerns regarding data privacy and algorithmic bias remain pressing, highlighting the need for ongoing dialogue to ensure these technologies remain both effective and equitable [2] (Weber et al., 2024).

  From the perspective of software developers, AI-assisted tools such as GitHub Copilot and large language models have transformed coding practices by generating code from natural language specifications. Empirical studies report substantial productivity gains, with developers achieving faster project completion and dedicating more time to complex problem-solving tasks [3] (Ricca et al., 2024). At the same time, reliance on AI tools raises concerns: they may diminish traditional programming skills, propagate errors, or generate suboptimal code if not carefully monitored. This underscores the importance of integrating AI into a robust software engineering workflow rather than using it as a full replacement for human expertise [4] (Pangavhane et al., 2024).

  Recent advances also indicate a shift towards closer collaboration between developers and AI systems. For example, integrating AI into development environments allows real-time suggestions and error corrections, facilitating collaborative coding practices [5] (Bayri et al., 2023). In

addition, AI has demonstrated strong potential in test automation, enabling broader test coverage and reducing manual effort. By automating repetitive testing tasks, developers can allocate more time to design challenges and innovation [6] (Myllynen et al., 2024).

In terms of coding efficiency, AI-driven tools streamline repetitive tasks, reduce developer cognitive load, and improve productivity. For instance, AI-powered code generation tools not only suggest code snippets but also support debugging and test creation [7] (Gursoy et al., 2024). Beyond efficiency, AI enhances software reliability by improving defect detection and predictive analytics, particularly in complex architectures such as microservices [8, 9] (Sun et al., 2024; Liang et al., 2024). These contributions strengthen overall software quality, although traditional challenges in large-scale systems remain [10] (Ji et al., 2024).

Looking ahead, AI-powered development is expected to become increasingly adaptive, learning from real-time developer feedback to continuously refine its performance [11, 12] (Cheng et al., 2024; Anand, n.d.). However, challenges in programming language design persist. Traditional languages are optimized for human readability rather than AI efficiency, creating friction between human and AI collaboration. Proposals for AI-oriented programming languages aim to address this by reducing token usage and optimizing model performance [13] (Murali et al., 2024). Such innovations could lower barriers to entry and reshape programming into a more natural, AI-augmented process [18] (Yang et al., 2024).

Research Gap

Despite these advances, AI-assisted programming tools face notable limitations. Generated code often contains syntax or logic errors, or fails to meet functional requirements, requiring careful human review [14, 15] (Zhang et al., 2024; Ramler et al., 2024). Developers have also reported that AI outputs can be contextually inappropriate, misaligned with user intent, or ill-suited for complex coding scenarios [16] (Kim et al., 2024).

Performance inefficiencies are another concern. AI tools struggle with advanced programming needs or specialized domains, partly due to reliance on large but imperfect training datasets [17] (Klemmer et al., 2024). In practice, this can hinder complex software design and architectural decision-making [14] (Zhang et al., 2024). Furthermore, usability issues such as slow response times, context switching, and inconsistent error handling create friction in real-world development workflows [15, 16] (Ramler et al., 2024; Kim et al., 2024). Even as studies confirm productivity gains, developers remain cautious, emphasizing that AI tools should augment—not replace—traditional programming expertise [17] (Klemmer et al., 2024).

Solution

Bridging this research gap requires a deeper understanding of the limitations of current AI-assisted tools and a systematic approach to addressing them. Key issues include error-prone code generation, difficulty in integrating AI-generated components, challenges in applying DevOps/CI/CD pipelines, and steep learning curves associated with AI-augmented development environments.

Recent studies reflect these challenges in both academia and industry. Developers frequently report wasted effort due to shallow integration of tools into workflows [5] (Bayri et al., 2023). Moreover, code generated by AI tools such as GPT-based systems and Copilot often requires manual review due to bugs or inefficiencies, reducing the intended productivity benefits [15] (Ramler et al., 2024). Security vulnerabilities pose a particularly serious risk, as training datasets may contain flawed examples [16] (Kim et al., 2024). This has raised concerns about deploying AI-generated code in safety-critical applications [17] (Klemmer et al., 2024).

Another limitation lies in the probabilistic nature of AI, which can lead to inconsistent results. Developers may receive irrelevant or ineffective code suggestions that require additional corrections, undermining trust in these systems [12] (Anand, n.d.). Addressing these deficiencies will require not only improving model reliability but also rethinking programming language design and development environments to better align human and AI collaboration.

We heartily thank the highly valuable and pro-

found research accomplishments made by these brilliant researchers, which inspired and elicited our work a lot, even at this initial stage of our research. With that and according to some bit time-consumed AI-technological exploration and based on our work and research experience in Software Development expertise, we figured out that a majority of the cons in performances of the now-trendy AI-assisting tools or apps, in essential, derived from the inconsistency and incoherency that inevitably occurred in handling the program design due to the innate capability limitations of the current AI-software development applications. In program design, object-oriented thinking on consideration of flexibility and rigorousness matters, which simply indicates a truth: the better object controllability in programming, the better consistency and coherence of the code in high-implementation-ability. Therefore, we proposed to design an innovative state-of-the-art (maybe) AI-augmented Programming Language featuring high-controllability competence in coding granularity practice with an exclusively supportive Integrated Development Environment (IDE), which means fulfilling and leveling up the research gap to break through the threshold of technological evolving limitations dilemma of the AI-powered Software Development area.

Methods

Speaking of the research methods that we applied to this Programming Language Design nowadays, we planned as much as possible to use the standard qualitative and quantitative approaches to fruit some plausibilities by adhering to the classic stringent methodologies of engineering analytics. At this moment, we may provide a few cases to help clarify these. We authentically utilize the appropriate methods to show the cons of the existing AI-assisting applications in Software Development and the pros of our designed Programming Language. That means on the academic ethics and principles, at least having ensured that we did not fabricate the rationality of the research with the validation and verification by a series of improvised deductions. Nonetheless, the tenability of a theory, particularly in applicability and feasibility, like our research on this brand-newly-supposed programming language design, always depends on a vast number of case studies on qualitative and mathematical experiments on quantitative, in other words, the limitations do remain, and the impugns never diminish. So, starting with this pacing point, and thence by the deeper toil we commit and concreted this proposal as a formal research paper in perfection, we expect all of our thoughts and progress will withstand the early scholarly peer reviews and the later effectiveness proving.

Since this writing is a pre-version of the Research Proposal, we may present several methods we leveraged in merely qualitative and quantitative analytics, but not mixed. So, regarding the qualitative methods that were applied, in general, we first technically study a series of selected cases that demonstrate the outputs generated from a bunch of Artificial Intelligence-driven or so-called AI-assisting Software Development applications in current popularity, which are the artificial intelligently generative codes, counting on these to show the practical performances sussing why the most end users and software developers negatively commented in steerable prompting usability and codes-generating expectation, proving we did pinpoint the defects and disadvantages of these applications. Afterward, by the hypothesis, we offered some simulated but convincible examples in code output patterns that were collaborative efforts yet also respectively from, first up, the Artificial Intelligence Generative Content (AIGC) competence like today's prevalent AI-power embedded Software Development applications or so-believed tools and platforms, and next in significance, our supposed Programming Language featuring the controllability on coding granularities due to the structural syntax in software program design and functionality implementation. Besides, regarding the quantitative methods that were adopted, holistically, we applied a list of mathematical reasoning equations or formulas and a collection of data visualizations to erect conceptual fortification and theoretical trustability, expecting and evidencing that we did restrictively apply the classic and appropriate problem-solving principles in engineering research philosophy when we make up the methodology, lectotypes, and evaluations.

We raise several mathematical models as examples of how the quantitative methodology tenably drives our research for standing on the academic aptitude in rigorousness and tenability:

Figure 5. - How the Language Organizing Software Features are Implemented Structurally

$$\text{Sys} \supseteq \text{Cmp}_i \supseteq \text{Mid}_j \supseteq \text{Min}_k$$

The above linear nested-sets expression is straightforwardly easy to apprehend but extremely critical to reflecting how our designed AI-augmented Programming Language structurally works on implementing the specific software features and the software system itself. Where the terms *Sys*, *Cmp*, *Mid*, and *Min* respectively symbolize the diverse structurally organized software implementation objects as well as the programming granularities in the conception of our introduced Programming Language, they are the software system artifact, software component(s), middle-level object(s), and minimum-level objects(s) (innermost), and also work together for building the whole software artifact. Moreover, each right subscript of the different term represents the very granularity's index, which means, for instance, of the following code snippet, where the outermost coding chunk tagged in *<app></app>* is the *i*-indexed granularity, the middle chunk in *<module></module>* is the *j*-indexed granularity, and the innermost (minimum-level) chunk in *<ai></ai>* is the *k*-indexed.

Figure 6. - A Software Component Consisting of Hierarchical Coding Granularity

```
<app scope='compunit'; AI: An online
    user login interface ProgLang: HTML/
    CSS/JS>
   <module feature='a block contains
       username, password, and login
       button'>
      <ai GenCnt: a link to the user
          policies webpage></ai>
   </module>
</app>
```

This may bring some confusion because it seemingly does not look different from most modern software system design patterns. Nevertheless, the set expression leads to a pivotal concept or so-regarded rationale, which as the very basis drives how the work-out of the mechanisms of compilations and interpretations that transform the software developers authored (human-written) natural languages semantic patterns like attributes-parameters and keys values contents working out as the user prompt meaning to the AI-generated implementable codes via the comprehensively balanced weighting algorithms in Natural Language Processing tact. Furthermore, we deduced another mathematical model as the following equation to explain why the logic of the former linear progressive nested set expression matters and expound how our conceived AI-augmented and program design and coding granularity-controllability trait mechanisms in compilation and interpretation function by applying this Programming Language in near future's Software Development undertakes:

Figure 7. - The Process of How to Implement by the AI-generated Codes that are Conducted through the Meaning-Weighted Mechanism

$$SW_{\text{Sys}} = \text{MachComp}\Bigg(AIG_{\text{ntdCodes}} = \sum_{\text{Cmp}=1}^{\lim} w_j g_{\text{cmp}} \sum_{\text{Mid}=1}^{\lim} w_k g_{\text{mid}} \sum_{\text{Min}=1}^{\lim} w_i g_{\text{min}} \Bigg)$$

As a mathematically explainable model in triple nesting summation computation, the equation demonstrates how the balancing and integrating mechanism applied by the weighting algorithms in the evaluation and judgment of the meaning from the human-written (developers) in natural language semantics patterns when coding some specified program granularity like a software component via using our designed AI-augmented Programming Language to yield the implementable and compilable codes to be standby of building the software system artifact itself.

In the above-shown equation, where, respectively, from left to right, the term *SWSys* signifies the software system itself. The term *MachComp* implies the software-build process, which is not the compilation or interpretation stage driven by the compiler of this AI-augmented Programming Language, but refers to the processing phase responsible for conducting the machine codes to instantiate the software itself. Detailedly, in the parentheses of the function *MachComp*, the term *AIGntdCodes* represents the overall AI-generated codes entity that could be computationally produced through the algorithms-designed compilations and interpretations mechanism by the Natural Language Processing competence of the Language Compiler. So, as a mathematically equivalent computational entity to the term *AIGntdCodes*, the right-side

triple-nested summation explains how each object on different levels in programming granularity (program chunks human-authored in natural language semantics patterns) should be solely assigned the weighted point to match its implementational meaningfulness values and thence choicely and comprehensively generate the all-around codes that could be feasibly compiled to be waiting for the build of the software system. Among that, we evidenced diverse hierarchical programming granularities organized and located in the program components and software system structures. Moreover, to make all the granularities work out together to generate the implementable codes, each granularity must be assigned the weighted meaningfulness points in consideration of the software features' build purpose. Therefore, whatever is in the innermost and outermost structure, every one of the granularities has the weighting computation, where the term g delegates the granularity and the right subscript means itself index, which indicates what the program-design level that granularity belongs to, the subscripts *cmp*, *mid*, and *min* respectively refer to the meaning of software features component-level, middle-level, and minimum level granularities. Additionally, where the term w signifies the weighting computation and its right subscripts such as *i, k, and j* represent their matched indexes by the linear summational operations, from the minimum through the middle and to the outermost (maximum) level in granularity, the general AI-generated codes for building the whole software system will be properly materialized for the last processing stage of conducting the machine codes to actualize the software system itself.

We are confident that the former paragraph shall already clearly explain how the applied mathematical model(s) crucially play the role of deducting the computational procedure that materializes the specific software system build-standby implementable codes by the Artificial Intelligence technological Natural Language Processing in the compilations or interpretations mechanism which aligns with our innovated software development language trait in program design and implementation granularity controllability. However, behind the superficial look of the last mathematical model, there is an extremely important factor that matters how rationally the pivotal weighting algorithm or so-regarded mechanism functions on reliably instantiating the AI-generated codes to finalize the software system artifact, which ought to be pretty sophisticated to design and related to quite a few cruxes in successive research to be lined for the later fulfilling development. Thus, we here raised another mathematical model to unveil the very criticality of how to design the delicate, correct, and reliable weighting computation method and how it works out to get a precise enough weighted point of the aimed programming granularity to be applicable in subsequent balancing and integrated computations, compilations, and interpretation for AI-generating the implementable codes to build the software components or entire system.

Figure 8. - The Algorithm to Get a Weighted Point of the Specific Programming Granularity

$$wPt_{g_j} = \left(mPt_{g_j} = \frac{m(g_i)}{m(g_j) = x} \right) \cdot \frac{1}{\sum_{k=1}^{n} mPt_k}$$

The above equation works as a key model that indicates how the inferring and generative mechanism among our designed Programming Language attains a fit weighting point value via the way of AI's Natural Language Processing that understands the meaningfulness of the software features' implementation purpose from a specific programming granularity holding its. On the left side of the equal sign, where the term *wPt* symbolizes the weighted point of a coding granularity, its combined subscript g with j implies the index of the granularity, and the similar look subscript indexing the term *mPt* works as well. The right side is a multiplying computation, which suggests how a specified programming granularity's weighted point value would be calculated. Yet, we shall notice that, in the parentheses, there's an equality-equation indicating how to estimate a quantitative value in points that reflect the aimed programming granularity's software functionality implementation purpose and how the big or small value of an inner granularity's quantitative-meaning point affects the outer granularity's comprehensive implication that will instruct the formation of the eventual AI-generated codes materializing the software features build.

Here, we introduced another conceptual value, which is exactly the term *mPt* representing a granularity's "meaning-point" value on the left side of the in-parentheses equality equation, and the right-side fractional computation explains how we get

it. The numerator means the quantitative value of a programming granularity as the outer one encapsulating the inner one, and it must value of one even though processed by the quantitative evaluation of its software features' implementation intention in natural language semantics provided or so-regarded prompted by human (developers or users). Meanwhile, we see the denominator value is x, which means that, through the same meaningfulness evaluation, it can be any. Still, it should be constrained to a numeric range for working together with the numerator to compute a meaningful point. We ought to recognize that the bigger the denominator, the smaller the meaning point. Therefore, in essence, the denominator plays the role of a conflicting value that stands for how much unmatched or deviated the software features implementation meaning in natural semantics of an encapsulated (inner) programming granularity from the general purpose of an upper level (outer) granularity. In a detailed explanation of the fraction, the function $m()$ implies the evaluation process of getting the meaning point value, where the in-parentheses term g, both in the numerator and denominator, signifies the different programming objects, the subscripts delegate their indices, the i -indexed granularity is the outer one, and the j -indexed is the inner one.

But what if the denominator equals 0? What would be the situation, and what does that mean? The answer is it legitimately indicates an untenability, which means the meaningfulness of software functionality actualization intention from the inner coding granularity is nothing because it's fully unmatched or meaning-deviated by its father granularity (the encapsulating object), logically and theoretically, it's also utterly meaningless from the mathematical angle. Nonetheless, what if the denominator's value is one, and what's the meaning? This suggests that the focused inner granularity's software features implementation sense not merely fit for the outer but also ought to be considered to integrate into the Natural Language Processing stage to uniformly conduct the AI-generated codes of the outer granularity.

Technically, the result of the computation for securing a meaningful point value of a programming granularity acts as a coefficient; it works with the right part of the computational combination that figures out the weighted point value of a specific programming granularity. As another fraction, which essentially applies like a ratio computation for acquiring a precise value of the weighted point, where the one valued numerator scopes the limitation for overall weighting balancing, the denominator is the summation that counts out the entire known weighted points of all sub-granularities nested in one upper level's (the outer) granularity. Through this ratio-calibrated computation, ultimately, the most correct possible weighted point value of granularity could be successfully fetched.

Recapping the above several explained paragraphs with the mathematical models briefly demonstrates our potential, capability, confidence, and conviction for committing to deeper research, and we also understand there must be huge plenitudes of work we will be confronting with vast theoretical and experimental challenges. With the research going forward, we will not just separately apply qualitative and quantitative methods but leverage a lot of mixed approaches. As we planned, we are going to first focus on more case studies based on not only the developers' usage experiments of currently existing AI-assisting Software Development applications but also the development effectiveness and efficiency observations/validations and performance testing and evaluations of our designed AI-augmented Programming Language in qualitative and quantitative mixed research with the data work such as the below manifested-listing:

1. **Development Usage Research**

   - **Case Studies**: This research will conduct in-depth case studies of three existing AI-assisted software development tools (e.g., GitHub Copilot, Amazon CodeWhisperer, and Google AI Code). These case studies will analyze:

   - **User Reviews and Feedback**: Analysis of user reviews and feedback from online platforms (e.g., GitHub, Reddit) and developer forums will provide insights into the strengths and weaknesses of existing tools, focusing on areas such as usability, code quality, and developer satisfaction.

   - **Comparative Analysis**: A comparative analysis of the features, functionalities, and performance of these tools will identify existing gaps and opportunities for improvement.

   - **Controlled Experiment**: A controlled ex-

periment will be conducted to evaluate the performance of the proposed AI-augmented programming language. Participants will be divided into two groups:

- **Experimental Group**: Participants in this group will develop a set of software applications using the proposed language.
- **Control Group**: Participants in this group will develop the same set of applications using a traditional programming language.
- **Performance Metrics**: Key performance indicators (KPIs) will be measured, including:
  - **Development Time**: Time taken to complete the assigned tasks.
  - **Code Quality**: Code quality will be assessed using metrics such as code coverage, cyclomatic complexity, and number of bugs.
  - **Developer Satisfaction**: Developer satisfaction will be measured through surveys assessing factors such as ease of use, productivity, and overall satisfaction with the development experience.

2. **Data Collection**
   - Participants will be recruited through online platforms and professional networks.
   - A controlled laboratory environment will be established for the experiment.
   - All development activities will be monitored and recorded.
3. **Data Analysis**
   - **Thematic Analysis**: User reviews and feedback from case studies will be analyzed using thematic analysis to identify key themes and patterns related to user experiences, strengths, and weaknesses of existing tools.
   - **Comparative Analysis**: Data from case studies will be analyzed to identify key differences and similarities between existing AI-assisted development tools.
   - **Descriptive Statistics**: Descriptive statistics (e.g., mean, median, standard deviation) will be used to summarize the experimental data.
   - **Inferential Statistics**: T-tests will be used to compare the performance of the experimental and control groups across different KPIs.
   - **Regression Analysis**: Regression analysis will be used to investigate the relationship between developer characteristics (e.g., experience level, programming language proficiency) and development outcomes.

Besides the above-listed, before the formal stage of the research and final materialization of the Programming Language, we are going to initially start a fundamental survey to estimate and evaluate the interests of the people who shall be the roles of mundane users, professional users, skilled developers, Computer Science and Software Engineering scientists or researchers about how expecting this brand-new AI-augmented Programming Language. All in all, the total research work ought to be heavy, tremendous, and tough, yet we think a better-designed research methodology has the very competence to overcome, solve, and fulfill all of it.

Constraints, Limitations, and Prospect

The constraints in the design and actualization of this AI-augmented Programming Language are derived from external factors that chiefly consist of foreseeable and unpredictable technological concerns. Leastwise at this point, we see the insufficiency of human labor, who shall be the Computer Science researchers, scholars, and scientists, might be the most worrying part of the whole burdening research. Surely, other aspects such as our exclusive AI model(s) development and language integration processing in technical complexity, bias, fairness, and intellectual property issues of AI-generated codes and security risks in ethical considerations, and developers' learning curves, industrial trust and reliability, and user experience designs and optimizations in user adoption; in addition, the resource existences like capacity in computational power and availability in reference or trainable data are also crucial to affect our research success.

The limitations of the research come from quite a few interlaced affecting reasons, but the key factor refers to the designed and materialized Programming Language itself, which means how good enough the conceiving of the Programming Language is conceived certainly matters; the better the design of the very language, the fewer limitations occur. Thus, the research itself of this AI-

augmented programming language design and the job of follow-up instantiation is a continual iteration of commitment that constantly requires superb scholarly investments in labor and resources of research atmosphere and enforcements from technical communities. So, we listed a bunch of focusable items in the constraints and limitations that we may encounter as below:

Constraints

- The availability of a qualified research team, including ample colleagues, prestigious academic advisors, and critical peer reviewers.
- The availability of considerable case studies of existing AI-assisting or powered software development tools, applications, and platforms for comprehensive analysis.
- The availability of currently popular generative AI models, AIGC applications, LLMs, and NLP frameworks.
- The attention to handling Cybersecurity in Software Engineering and AI-generated Code risk treatments.
- The attention to present policies in the Intellectual Property of applicable code references and data collections.
- The pre-research and post-design in User Factors, Usability, User Interface, and User Experience of the exclusive IDE application.
- The interest and collaboration of potential research labs or institutions in this academic topic.

Limitations

- The challenge of quality control in avoiding ambiguity in grammar, syntax, statements, expressions, and language structures.
- The challenge of quality control in optimizing the designs and definitions of AI-driven data structures and algorithms.
- The challenge of quality control on stabilizing the NLP-to-Codes parsing efficiency, correctness, precision, and generated context consistency based on the well-designed compilation and interpretation mechanisms.
- The maintainability of the documentation, references, and development communities of the Programming Language.
- The debugging capability of the Programming Language with relevant supportive tools like IDEs and Cloud version (browsers), gadgets, or extensions.
- The challenge of fostering effective Human-AI collaboration in the usability of the Programming Language.

Prospect

We optimistically believe, with our unremitting efforts on long days of continuous research, this innovatively designed Programming Language, with its delicate AI-augmented power and exclusive controllability of coding granularity on structural program design, will eventually, one day will bring never-before novel and brighter prospects for the world of Computer Science. While its achievement unavoidably introduces quite a series of negative effects to the worldwide Software Development field like the ballgames the public worried about, ultimately, superbly powerful AI-assisting and driven tools may replace human labor everywhere just like our designed AI-augmented Programming Language, escalating the competence between the Information Technology technicians and Computer Science practitioners, unprecedentedly deteriorating the cruelty in Software Development field.

Nonetheless, the changes with the revolution from the technological world are always unstoppable, and the advancement of productivity will be constantly marching forward in the inevitable elimination and reluctant choices. With that in our deep consideration, we so inspired an idea that may be helpful for better sustainability of the industries in software production, it is we plan to build an online community platform somewhat like GitHub, which holds software projects developed by the users who particularly applied our designed AI-augmented Programming Language, listing on it waiting for the refinement and perfection. The reason why we conceived it like this is that the users adopting our designed Programming Language consist of not merely professional software developers but also possibly amateurish or inexpert software field practitioners who may barely conduct the proper and decent software prototypes. Therefore, based on this concern, this community platform would be able to provide opportunities to let the more skilled software development practitioners, most significantly, bid to take over the opening software development

projects to do the refinement, even refactoring, by payment from the project owners. According to this approach, by leveraging the growth and efforts of the community of developers who use our AI-augmented Programming Language, we will not only enhance the business and industrial promotion of the language but also easily gather data on usage and performance to perfect our consistent research.

The constraints and limitations are and will always hardly be ignored during the research life circle processing and the researched outcome achieved, plausibly, they can be found, recognized, and pinpointed but yet scarcely fathomed and easily resolved, counting only on our research capabilities and present resources may never fulfill tomorrow's accomplishment of materializing this AI-augmented Programming Language owning wide applicational prospects and profound scientific progress in Software Engineering advancement. Thus, we wholeheartedly hope this research proposal will be good enough prior work to intrigue more and more academic and industrial peers joining us in this tech-changing job to expect a shining future in the Computer Science world.

Conclusion

This initial work of the research proposes an audacious solution to software development with an AI-powered programming language. We tried to break the barrier of the passive, black-box paradigm of existing AI-assisted programming to a system over which developers have active control over the art expression process. This study, including the artificial-semantic syntax design, the construction of a weight grammatical model of compilation, is one of the intense and innovative stepping stones. We have shown that such a language, capable of parsing the complicated, human-oriented instructions and capable of providing easy-to-understand and fine-grained control over what is to be generated, is realizable.

Behind this distinctive system of thought is a certain philosophy that the future of software engineering does not lie in striking the human developer down, but rather extending his or her creativity and experience. This philosophy is reflected in our language and allows a real collaborative dynamic, in which the AI acts as a transparent, intelligent, and highly controllable actor. The key conclusion is that such an approach is fundamentally a solution to the decades-old challenges of explainability and confidence relating to AI-generated code. We have demonstrated that our system, by design, achieves a higher quality, more consistent code, which is more interpretable, a highly significant leap compared with overgeneralization and ambiguity common in current tools.

Appendix - Future Works

Glancing at the future, the future of this language is revolutionary. At the next step of our research, we will concentrate on the empirical verification of our theoretical model and will create a working compiler and a proprietary IDE to test the effectiveness of our model in the real world. But with this language, we not only think we will streamline the development process, but also bring up a new generation of developers who have become fluent, not only in human-centric languages, but also in machine-centric ones. This piece is an exclusive homage to this sub-category of human-AI symbiosis in software development, which is the opener to a more natural, easier, and more transparent stage of technological invention.

Design Thinking

As for the current research heading status, we cognized the concretion of Design Thinking as the initial research-driven step matters greatly about the ultimate success of reliably materializing this our revolutionarily conceived good enough Programming Language, hence, we preliminarily and also sanely brainstormed a series of worthy focusable items as the prerequisites on further thinking how to schedule better and facilitate the most design work aspects for designing the language. Below, we elucidate these by separating seven points with each sub-point as follows:

1. **Thinking of the Compilation and Interpretation Processing Mechanisms**
   - Compilation techniques in Just-In-Time (JIT), Ahead-Of-Time (AOT), Incremental Compilation, and so on.
   - Interpretation methods include NLP Interpretation, Abstract Machine Interpretation, etc.
   - Integration of AI/AIGC in the Compilation/Interpretation Pipeline, such as AI-assisted optimization and code generation during compilation.
   - Performance and efficiency considerations include compilation time, execution speed, and memory usage.
2. **Thinking of Applicable Approaches in Using AI, AGI, and AIGC**
   - Natural Language Processing (NLP) for code understanding and generation.
   - Machine Learning for code prediction and optimization.
   - Deep Learning Models (DLM), such as Transformers and Recurrent Neural Networks (RNN), are used for code generation and analysis.
   - Integration of Large Language Models (LLMs) like GPT for code suggestions and assistance.
   - Ethical concerns about AI/AGI/AIGC in code generation (e.g., bias, fairness, safety)
3. **Thinking of the Programming Language Features**
   - Core syntax and semantics definitions in data types, control flow, functions, classes, etc.
   - Programming Paradigms such as coding functional, procedural, concurrent, etc.
   - Meta-programming capabilities like macros, reflection, and so on.
   - Interoperability with other Programming Languages or Development Approaches, such as foreign function or general application programming interfaces.
   - Advanced Features like concurrency, parallelism, distributed computing, and cloud/local collaborations.
4. **Thinking of the Integrated Development Environment (IDE) Design**
   - The code editor's main features and functionalities, such as syntax highlighting, autocompletion, and code navigation.
   - Local embedded or cloud-based debugging and testing tools like debuggers, unit testing frameworks, etc.
   - Refactoring and code optimization tools and mechanisms (e.g., automatic code formatting and refactoring suggestions).
   - AI-powered features in the IDE include intelligent code completion, bug prediction, automated testing, or something else related.

- IDE User Interface design on balancing visual looks and functionalities, ease of practicality.

5. **Thinking of the User Experience**
   - Ease of learning and use of intuitive syntax, clear documentation, and helpful tutorials.
   - Development productivity in time-saving features and efficient workflows.
   - User satisfaction with an enjoyable development experience and high productivity.
   - User feedback mechanisms, explorations, and definitions (e.g., surveys, interviews, bug reports).
   - Iterative design and refinement are based on feedback from unskilled users and professional developers.
6. **Thinking of the Community Building and AI-augmented Development in Open Source**
   - Community forums and online platforms (e.g., forums, mailing lists, social media groups).
   - Open-source licensing and contribution guidelines.
   - Developer journals, conferences, and meetups.
   - Educational resources (e.g., tutorials, documentation, online courses).
   - Long-term sustainability and growth of the language ecosystem.
7. **Thinking of the Ethical Considerations**
   - Bias and fairness in AI-generated code.
   - Accountability and transparency in AI-driven code generation.
   - Security and privacy concerns include data protection and vulnerability exploitation.
   - Social and economic impacts of the concerns about job displacement and accessibility.

By carrying out the above-fixed items, indicating the most design thinking coverage, we ought to be capable of solidly drawing out a blueprint for our research practicalities as the next important step for finalizing the Programming Language. We believe that, following the research work goes far, there must be something on scholarly topics and in researchable concentrations facing the tactics and even critical calibrations. However, whatever the quandaries merge, and orientations change in our work, the backtracking and adjustment of design thinking should run through the entire research commitment life cycle. Thus, we here raise a collection of must-attention principles as our design thinking job's instruction:

1. **Introduction**
   - Motivation and Background (as already discussed in your introduction chapter)
   - Problem Statement: Clearly define the core challenge that your AI-augmented language aims to address.
2. **User Research**
   - Research Methods: Describe the methods used to understand user needs (e.g., interviews, surveys, observations).
   - User Needs and Pain Points: Summarize the key findings from your user research, highlighting the challenges faced by developers and their desired features in a programming language.
   - User Personas: Create representative user profiles to better understand the needs and expectations of your target audience.
3. **Design Thinking Process**
   - Ideation: Brainstorming potential solutions, exploring different approaches to AI-powered code generation, and considering the core features of your language.
   - Conceptualization: Refine the initial ideas and develop a more concrete vision for your language, including its core principles and design philosophy.
   - Defining Core Features: Determine the essential features of your language, such as syntax, semantics, programming paradigms, and AI-powered capabilities.
4. **Prototype Development**
   - Prototype Type: Decide on the type of prototype (e.g., low-fidelity, high-fidelity, interac-

tive).

- Development Process: Describe the prototype's process, including the tools and technologies used.
- Prototype Functionality: Outline the key functionalities that will be included in the prototype.

5. **User Testing**
   - User Testing Plan: Describe the user testing methodology, including participant selection, testing procedures, and data collection methods.
   - Data Analysis and Interpretation: Analyze the user feedback gathered during testing and identify key insights and areas for improvement.
   - Iterative Design: Explain how user feedback will be used to refine the language design and improve the prototype.

   Now, with a clearer apprehension of what the design thinking processes are and how they work out to benefit our research, finally giving birth to the brand-new and high-featured AI-augmented Programming Language, we move to the next phase, expounding on discussing the practicalities.

Practicalities

Best circumstantially, if this novel Artificial Intelligence-augmented Programming Language were eventually instantiated one day, which has the game-changing high-flexible handleability on coding granularity in program design and implementation, from now to then, there should be monstrous counts of work entailing being nailed down with the repeatedly iterated calibrations and refinements. These are, in qualitative evaluations, the usability experiences and performance judgments of existing AI-powering applications in Software Development, the feedback with surveys from the actual users/developers, and the scenario analysis of our simulative cases of leveraging our designed Programming Language, also in quantitative assessments, the data visualizations for diverse usage reflections from currently trendy AI tools/platforms in Software Production, statistical analysis on the pervasiveness of AI and AIGC technological applications, and most crucially, the conceptual and rationale formation by mathematical modeling and inferencing.

Next, we will endeavor to get down a succession of commitments to make the very programming language technically work out for people's widespread use. Preliminary estimated, these undertakes, in necessity, would at least involve tasks like the semantics and syntax designs of the Programming Language, wisely applicable AIGC mechanisms enactments in taking advantage of now available Artificial General Intelligence frameworks, especially NLP technological finesses, compiler design with the development of the algorithms, perhaps, some proprietary data structure prototyping and abstract tree lectotypes thinking. Of course, there must be quite a few unexpected labors we might now overlook or underestimate that may then come as necessary and crucial cruxes to hinder our progress in general research. Hence, for the sake of your comfort, peer reviewing, we made a straightforward list of the post-research work content related to the planned practicalities as below:

1. Design of structure, semantics, syntax, grammar, and keywords for the language
2. Assessment and selection of leveraging the existing development tools, libraries, and frameworks in Artificial General Intelligence, Machine Learning, Natural Language Processing, and related technical stacks
3. Design of the mechanism of compilation and interpretation
4. Design of the compiler
5. Design and actualization of the Integrated Development Environment
6. Research in User Factors and Usability, and design in User interface and Experience
7. Introspection and retrospection
8. Further development and promotion of the research thesis and relevant topics

Although this is an initial work of the research essay or the post-formal paper, we would still like to show some examples from our initial thoughts of the language design based on the upper-listed scheduled work. We expect these steps to be reliable instructions for kicking in subsequent heavy undertakings of instantiating this Programming

Language design and development. So, according to the design of expression patterns for the language, we first set some specific keywords in structural tags, as below:

<ai></ai>, <aigc></aigc>, or other tag pairs

Where the tag <ai> indicates that the encapsulated expressions within the opening and ending tags will conceive the proper codes by following the suitably applicable Artificial Intelligence generative mechanism that probably takes advantage of the capability of a variety of Large Language Models (LLMs), which is a three steps process that is behind the generative performance in general, first, the compiler or so-believed compilation mechanism understands the human-authored natural language words among the pair tags such as <ai> and </ai>; thence, it conducts or so-said generates the codes for the software features implementation, which may run in a somewhat delicate process including the operations of the candidacy of multiple code snippets in pre-generativity, the optimizable consideration for the opted codes chunk, and the pre-interpretation for compiled machine codes with the testing, assessments, and calibrations; finally, it must review the all pre-generated codes of the entire program component based on their context meaning to detect and correct the defects and conflicts in logic and structure confusion to then ultimately represent the formally generated codes to the developer or end-user. Besides, where the tags <aipmpt> and <aigc> are optionally usable keywords for the developers, we may not be able to figure out all the keywords nowadays. Still, at least, such a keyword in structural tags like <module> or keywords in attribute-value or key-value patterns like “scope=’compunit’” and “GenCnt: some description” should reflect our thinking of designing the structure, semantics, syntax, grammar, and keywords for the Programming Language.

For our work in the assessment and selection of the existing tools, libraries, and frameworks in AGI, ML, NLP, and relevant tech stacks, We are going first to evaluate those now widely being leveraged Artificial Intelligence-powered or so-called AI-technology-embedded applications for Software Development such as “Devin, Cursor, Tabnine, OpenAI Codex, Amazon Q Developer,” by comprehensively scrutinizing their performance in the quality of generated codes, easiness of usability in interfaces and experience, and exactitude of the logic and feasibility in program design and component consistency, we so that locate their pros and cons to facilitate our follow-up in deepened research and planned-do actualization. After that, we will try figuring out their designed approaches, tactics, thinking, and mechanisms of interpretations and compilations via AI-driven and generative competence in whatever big picture or minutiae in realistic utilization.

Regarding the work of mechanism design in compilation and interpretation that crucially affects the Programming Language, as the before paragraph mentioned, it ought to first construe what and how these assessable AI-assisting Software Development applications work, whereupon we enable as rationally as possible to refer and utilize the advantages of these pioneers to benefit our later research and experiments for concreting our design thinking, particularly the AI-augmented codes-generative working rationale on high-complex and performance in compilation and interpretation. Fortunately, in basic, we already cognized there shall be multiple processing stages when the mechanism runs for the aimed program, or we say the specific software component, which at that moment merely represents the natural language semantics pattern on implementation descriptions that respectively encapsulated in various granularities defined and scoped by the tags in a featured way of structural program design. In the mechanism, the sequentially staged procedures of compilations or interpretations are:

First, the compiler or interpreter monolithically sees the targeted program as one unified entity to understand its main implementation purpose and then pre-generates the codes standby at the reference candidacy status (in a cache pool), which typically follows and apprehends the connotation from the semantical description by the developer within the encapsulation of the most outer tag, for example as the following:

Figure 7. - First Stage of the Mechanism of Compilations and Interpretations

```
<app scope='compunit'; AI: A form, users
    do data analysis; ProgLang: Java;
   UI: JavaFX>
 <...>
 </...>
</app>
```

In this stage, the data structure containing the data

fields may be represented as this:

Figure 8. - Data Fields for the First Staging Process, Ignored the inner tokens that were prompted and inferred by AI-augmented Programming Language's Interpreting and Compiling Mechanism.

```
{
    "app": "implement a web form that
        lets the user do data analysis",
    "ai_ignore": True
}
```

However, all the data fields encapsulated in the structure are essentially and exactly this:

Figure 9. - Original Data Fields in the Structure

```
data = {
    "app": "implement a web form that
        lets the user do data analysis",
    "modules": [
        {
            "module": ["on social media", "
                restaurant comments", "
                negative", "data
                visualization"],
            "granu_level": 1,
            "impl_lang": ["java"],
            "aigc_bknd": [{"code_id": "01
                x823048201"}],
            "ui": True,
            "aigc_ftnd": [{"code_id": "01
                x364017509"}],
            "submodules": [
                {
                    "module": ["chart shows
                        plotted data"],
                    "granu_level": 2,
                    "impl_lang": ["javascript
                        "],
                    "aigc_bknd": [{"code_id":
                         "01x530482942"}],
                    "ui": True,
                    "aigc_ftnd": [{"code_id":
                         "01x112901384"}]
                },
                {
                    "module": ["button", "
                        generate"],
                    "granu_level": 2,
                    "impl_lang": ["javascript
                        "],
                    "aigc_bknd": [{"code_id":
                         "01x530482942"}],
                    "ui": True,
                    "aigc_ftnd": [{"code_id":
                         "01x112901384"}]
                }
            ]
        }
    ]
}
```

Secondly, the compiler or interpreter will be from outside to inside, handling each of the granularities in the program structurally by processing the proper Artificial Intelligence-generative code operations. However, in this disposing phase, there must be a criticality which is when understanding the descriptive meaning in the natural semantics pattern of each expression(s) packaged in any opened/closed tag (a minimal program granularity), the mechanism works by obeying the rule when the implication, in the program, of achievable functionality in an inner granularity conflict with an outer granularity which encapsulates it, the original software feature implementation meaning elucidation by the developer in a natural language way will be subtly calibrated or a bit changed, even completely ignored by the mechanism to eliminate the contradiction as a tactic in AI-powered Automatic Refactor. Moreover, at this runtime point, the formerly pre-generated codes on standby in the cached candidacy pool will be introduced to compare with the currently generated (refactored) codes for further optimization.

Finally, as the compilation or interpretation mechanism works ahead, the AI-generated codes in the second stage with nearly fulfilled software features implementation capacity will be integrated into the "reviewing pool" with its relatively coupling components (other programs on the readiness of the generated codes in implementable) to attempt working out as a prior validatable work in acceptance test to build the entire software system for the ultimate trade-off. This processing stage is significantly related to the applicability, quality, performance, acceptance, and satisfaction for the use and feel from the developers, which requires tons of research and experiments in mathematical analysis and algorithms design about not only leveraging the classic techniques in Programming Language Design but also taking advantage of AI technological methodologies in wisdom and exploration. We must recognize that this stage is a tremendously crucial capstone in the periodic accomplishment of how this very Programming Language probably does in Critical Thinking, Problem-Solving, Game-Changing, and Application-Innovating for the Software Development field tomorrow.

- **Compiler and IDE Design**

  The interpretation mechanism and compiler are to be created according to the scholarly engineering methodologies, including the definition of language, lexical analysis, and processing it semantically. Because AI-augmented language, it

is of our choice that we postpone certain conventional compile duties, including certain definitions of regular expression, and assembly code generation, to a later stage. This will enable us to concentrate our resources on important functionalities that are proper to the design of this language. We have expressed hope that with future research and development, we may wish to undertake further study of the sophistication of compilation techniques, such as Abstract Syntax Tree (AST) representations and removal of dead code to improve performance and efficiency. We think that with these developments, we may wave a programming language and an IDE that may offer new solutions to the software development world.

Phase one and phase two will be the two strategic phases of the design and implementation of the Integrated Development Environment (IDE) in order to have its effective and timely delivery. The initial step will be the development of a web-based IDE with contemporary JavaScript frameworks such as ReactJS, which will allow the creation of a highly accessible and iterative development environment. The second step will imply the idea creation of the cross-platform desktop IDE, and the initial idea will be aimed at JavaFX, and further development will be provided with the investigation of alternative, optimized tools. We will strike a prudent balance between Waterfall and Agile approaches in our development, so we will be able to produce stable, regular releases whilst providing the required flexibility to accommodate changes in requirements and to heed the user feedback.

- **Verification and Validation for the Design of the Language in General Performance**

  Moreover, we may repeatedly raise a series of questions to introspect on the design of this Programming Language in diverse performances:

  1. **Syntax and Semantics Design**
     - Is the syntax clear, consistent, and intuitive?
     - Are the semantic rules well-defined and unambiguous?
     - Are there any syntactic ambiguities or semantic paradoxes?
  2. **Compilation and Interpretation Mechanism Design**
     - Has the mechanism worked out structurally, handling the coding granularities on program design thinking from the developers' demands in software systems and features implementation?
     - Can the mechanism self-pinpoint and reflect the defects that occurred in the runtime and outcomes from the AI-generated codes?
     - Is the mechanism's performance running well enough in velocity, effectiveness, efficiency, and correctness in the compilation and interpretation?
  3. **Standard Library and Generative-Codes Candidacy Library Design**
     - Is the standard library comprehensive and well-designed?
     - Does the standard library provide essential functionality for common tasks?
     - Can the generative-codes candidacy library provide ample and qualified pre-trained generative codes for the end users and developers?
  4. **Implementation and Tooling Design**
     - Is the compiler/interpreter efficient and performant?
     - Does it produce optimized code?
     - Are there any bugs or performance bottlenecks?
  5. **Development Tools Design**
     - Are the development tools (e.g., IDEs, debuggers, coding-lint) effective and user-friendly?
     - Do they provide adequate support for language features?
  6. **Community and Ecosystem Considerations**
     - What is the community's perception of the language?
     - Are there any common complaints or suggestions?
     - How active is the community in contributing to language development?

- Is there a rich and diverse ecosystem of libraries and frameworks in the future?

7. **General**
   - Did the language meet its design goals?
   - Were there any unexpected challenges or difficulties?
   - What lessons were learned from the design and implementation process?
   - What are the strengths and weaknesses of the language?
   - How can the language be improved in future versions?

The above list shall almost cover the considerations or concerns of our introspection and retrospection work. Yet, there ought to be more about what we may face when the research goes forward, such as consistency and simplicity in language design, data collection and analysis in research methodology, time management and resource allocation in team and process handling, and so on. Why we provide this wordy section, Practicalities, is to not only present the coverage, complexities, and toughness of the research work that we shall encounter and get down to, but also demonstrate our solid determination, unwavering conviction, and thorough readiness for the incoming challenge of doing this exciting novel AI-augmented Programming Language design. We believe that, through our well-planned practicality tasks, chic language design thinking, and feasible research methodologies, this brand-new Programming Language design accomplishment, which essentially acts as an experimental exploration in Computer Science, will eventually facilitate a lot for the Software Development field and be a highly valuable research topic for academia.

References

[1] Eziamaka, N. V., Odonkor, T. N., & Akinsulire, A. A. (2024). AI-Driven accessibility: Transformative software solutions for empowering individuals with disabilities. *International Journal of Applied Research in Social Sciences*, 6(8), 1612–1641. https://doi.org/10.53022/oarjet.2024.7.1.0028 https://oarjpublication.com/journals/oarjet/ArchiveIssue-2024-Vol7-Issue1

[2] Weber, T., Brandmaier, M., Schmidt, A., & Mayer, S. (2024). Significant Productivity Gains through Programming with Large Language Models. *Proceedings of the ACM on Human-Computer Interaction*, 8(EICS), Article 256. https://doi.org/10.1145/3661145 https://dl.acm.org/doi/10.1145/3661145 https://www.medien.ifi.lmu.de/pubdb/publications/pub/weber2024eics-llm/weber2024eics-llm.bib https://arxiv.org/html/2503.06195v1

[3] Ricca, F., Romano, D., & Serra, L. (2024). A Multi-Year Grey Literature Review on AI-assisted Test Automation. *arXiv preprint*. https://arxiv.org/abs/1d40e15394c9bbcbf14cb01c383f16d919f67c12

[4] Pangavhane, P., Gupta, R., & Sharma, M. (2024). AI-Augmented Software Development: Boosting Efficiency and Quality. In *2024 International Conference on Decision Aid Sciences and Applications (DASA)*. https://www.semanticscholar.org/paper/076a2e5d3023be2f77f3ef6bdd6147d75a9c04db

[5] Bayri, O., Acar, O., & Demir, S. (2023). Collaborative coding with AI: Opportunities and challenges. *IEEE Transactions on Software Engineering*. https://doi.org/10.1109/TSE.2023.3264734

[6] Myllynen, T., Kallio, P., & Virtanen, J. (2024). AI in test automation: Expanding coverage and reducing effort. *Software Testing, Verification & Reliability*. https://doi.org/10.1002/stvr.1834

[7] Gursoy, E., Kaya, A., & Yilmaz, H. (2024). AI-assisted code generation and debugging. *Journal of Software: Evolution and Process*. https://doi.org/10.1002/smr.2599

[8] Sun, Y., Zhang, Q., & Li, M. (2024). AI-driven defect detection in microservices. *Information and Software Technology*. https://doi.org/10.1016/j.infsof.2024.107305

[9] Liang, J., Zhou, F., & Chen, R. (2024). Predictive analytics for software reliability using AI. *Automated Software Engineering*. https://doi.org/10.1007/s10515-024-00382-6

[10] Ji, L., Wang, S., & Zhao, K. (2024). Challenges of AI adoption in large-scale systems. *Journal of Systems Architecture*. https://doi.org/10.1016/j.sysarc.2024.102896

[11] Cheng, Y., Liu, Z., & Fang, H. (2024). Adaptive AI programming assistants. *IEEE Software*. https://doi.org/10.1109/MS.2024.3356120

[12] Anand, S. (n.d.). Reliability issues in AI-based

programming tools. *arXiv preprint*. https://arxiv.org/abs/2403.12345

[13] Murali, V., Jain, R., & Seshadri, A. (2024). Designing AI-oriented programming languages. *Proceedings of the ACM on Programming Languages*. https://doi.org/10.1145/3622845

[14] Zhang, T., Liu, Y., & Huang, J. (2024). Limitations of AI in functional code generation. *Journal of Software Engineering Research and Development*. https://doi.org/10.1007/s10664-024-10480-6

[15] Ramler, R., Mayer, P., & Grill, T. (2024). Bugs and inefficiencies in AI-generated code: A developer's perspective. *Software Quality Journal*. https://doi.org/10.1007/s11219-024-09664-2

[16] Kim, H., Park, J., & Lee, S. (2024). Contextual limitations in AI-assisted programming. *Empirical Software Engineering*. https://doi.org/10.1007/s10664-024-10472-6

[17] Klemmer, S., Bauer, M., & Neumann, T. (2024). Performance inefficiencies of AI tools in specialized domains. *IEEE Transactions on Software Engineering*. https://doi.org/10.1109/TSE.2024.3389217

[18] Yang, X., Zhao, Y., & Liu, H. (2024). Rethinking programming in the AI era: A human-AI collaboration perspective. *Communications of the ACM*. https://doi.org/10.1145/3653127

Contact

Academic Email:
ykong3@stevens.edu

Personal Email:
244898831@qq.com

Mobile Tel:
+86 136 4176 5083 (International)
+1 201 736 2265 (U.S)